pgBadger for PostgreSQL

Step-by-Step Procedure: Install, Configure, Run and Analyse

pgBadger is a fast, open-source PostgreSQL log analyzer written in Perl. It reads your PostgreSQL log files and produces detailed HTML reports covering slow queries, most frequent queries, errors, connections, locks, temporary files, checkpoints, autovacuum activity and more.

Contents

- 1. How pgBadger works
- 2. Prerequisites
- 3. Install pgBadger
- 4. Configure PostgreSQL logging
- 5. Generate logs and locate them
- 6. Run pgBadger (first report)
- 7. Useful command-line options
- 8. Incremental reports with cron
- 9. Read and interpret the report
- 10. Troubleshooting
- 11. Best practices and quick reference

Note: Commands below are for Linux. Paths and service names vary by distribution and PostgreSQL version; adjust them (for example, `16` in a path to your version).

1. How pgBadger works

pgBadger does not connect to the database. It parses the text log that PostgreSQL writes, so the quality of the report depends entirely on what PostgreSQL is configured to log.

- PostgreSQL writes statements, durations, errors and events to its log.
- pgBadger parses that log (plain, compressed, stderr, syslog, csvlog or jsonlog).
- It outputs a self-contained HTML report you open in any browser.

2. Prerequisites

- A running PostgreSQL server and shell access to the server (or a copy of its logs).
- Permission to edit `postgresql.conf` and reload PostgreSQL (superuser or OS admin).
- Perl 5.8 or later (present on almost all Linux systems).
- Enough disk space for logs. Logging every query (`log_min_duration_statement = 0`) can generate very large files on busy systems.

3. Install pgBadger

Option A: Package manager (simplest)

```
# Debian / Ubuntu
sudo apt update
sudo apt install pgbadger

# RHEL / CentOS / Rocky / Alma (PGDG or EPEL repository enabled)
sudo dnf install pgbadger
```

Option B: From source (latest version)

```
sudo apt install perl libjson-xs-perl make wget   # or: dnf install perl-JSON-XS make wget
# Download the latest release from https://github.com/darold/pgbadger/releases
tar xzf pgbadger-X.Y.tar.gz
cd pgbadger-X.Y
perl Makefile.PL
make && sudo make install
```

Verify the installation

```
pgbadger --version
pgbadger --help
```

4. Configure PostgreSQL logging

Step 4.1: Find postgresql.conf

```
sudo -u postgres psql -c "SHOW config_file;"
```

Step 4.2: Edit the logging parameters

Add or change the following in `postgresql.conf` (or use `ALTER SYSTEM`):

```
# --- Where and how to log ---
logging_collector = on
log_directory = 'log'                    # relative to data dir, or absolute
log_filename = 'postgresql-%Y-%m-%d.log'
log_rotation_age = 1d

# --- Required prefix so pgBadger can parse lines ---
log_line_prefix = '%t [%p]: user=%u,db=%d,app=%a,client=%h '

# --- Queries ---
log_min_duration_statement = 250         # ms; 0 = log all, -1 = disable
log_statement = 'none'                   # do NOT combine with duration logging
log_duration = off

# --- Events pgBadger reports on ---
log_checkpoints = on
log_connections = on
log_disconnections = on
log_lock_waits = on
log_temp_files = 0
log_autovacuum_min_duration = 0

# --- Keep messages in English format ---
lc_messages = 'C'
log_error_verbosity = default
```

Parameter	Purpose
log_line_prefix	Gives pgBadger timestamp, PID, user, database, application and client. Must match the `-p` option if you use a custom prefix.
log_min_duration_statement	Logs statements slower than the value (ms). Use 0 only for short, controlled periods.
log_statement	Leave as 'none'. Using it together with duration logging duplicates every query.
log_checkpoints / log_temp_files / log_lock_waits / log_autovacuum_min_duration	Feed the checkpoint, temporary file, lock and autovacuum sections of the report.
lc_messages = 'C'	Ensures messages are in English so pgBadger can recognise them.

Step 4.3: Reload PostgreSQL

Most of these settings only need a reload. Changing `logging_collector` requires a full restart.

```
sudo -u postgres psql -c "SELECT pg_reload_conf();"
# only if logging_collector was changed:
sudo systemctl restart postgresql
```

Step 4.4: Confirm the settings

```
sudo -u postgres psql -c "SHOW log_line_prefix;"
sudo -u postgres psql -c "SHOW log_min_duration_statement;"
```

Tip: If you use syslog, set `log_destination = 'syslog'` and run pgBadger with `-f syslog`. For CSV logs set `log_destination = 'csvlog'`; in this case no `log_line_prefix` is needed.

5. Generate logs and locate them

- Let the database run through a representative workload (peak hours, batch jobs).
- Find the log directory: `SHOW data_directory;` then look in the `log` folder, or check `/var/log/postgresql/` on Debian/Ubuntu.

```
ls -lh /var/lib/postgresql/16/main/log/
# or
ls -lh /var/log/postgresql/
tail -n 20 /var/log/postgresql/postgresql-16-main.log
```

You should see lines containing timestamps, the prefix and `duration: ... ms statement: ...`.

6. Run pgBadger (first report)

Step 6.1: Basic report from one log file

```
pgbadger /var/log/postgresql/postgresql-2026-10-28.log -o report.html
```

Step 6.2: Custom log_line_prefix

```
pgbadger -p '%t [%p]: user=%u,db=%d,app=%a,client=%h ' \
         /var/log/postgresql/postgresql.log -o report.html
```

Step 6.3: Several or compressed logs, in parallel

```
pgbadger -j 4 /var/log/postgresql/postgresql-*.log* -o report.html
```

Step 6.4: Restrict to a time window

```
pgbadger -b "2026-10-28 09:00:00" -e "2026-10-28 18:00:00" \
         /var/log/postgresql/postgresql.log -o office_hours.html
```

Step 6.5: Open the report

```
xdg-open report.html                      # Linux desktop
scp user@dbserver:~/report.html .         # copy to your laptop, then open it
```

The report is a single HTML file, so it can be emailed, archived or served by any web server.

7. Useful command-line options

Option	Description
-o FILE	Output file (.html, .json, .txt, or .bin). Default is out.html.
-O DIR	Output directory (required for incremental mode).
-f FORMAT	Log format: stderr, syslog, syslog2, csv, jsonlog, etc. Usually auto-detected.
-p PREFIX	The log_line_prefix used in postgresql.conf.
-j N	Use N CPU cores to parse files in parallel.
-J N	Parse N log files simultaneously.
-b / -e	Begin / end datetime filter, format YYYY-MM-DD HH:MM:SS.
-d DB	Report only on the given database.

Option	Description
-u USER	Report only on the given user.
-t N	Number of top queries to display (default 20).
-T TITLE	Custom report title.
-I	Incremental mode (see next section).
--retention N	Weeks of incremental reports to keep.
-q	Quiet mode; no progress output (good for cron).

8. Incremental reports with cron

Incremental mode creates a daily report per day plus a weekly summary and an index page, only reprocessing new log data each run. This is the usual setup for production.

Step 8.1: Create the output directory

```
sudo mkdir -p /var/www/html/pgbadger
sudo chown postgres:postgres /var/www/html/pgbadger
```

Step 8.2: Test it manually

```
sudo -u postgres pgbadger -I -q -j 2 \
     -O /var/www/html/pgbadger \
     /var/log/postgresql/postgresql-*.log
```

Step 8.3: Schedule it

```
sudo -u postgres crontab -e

# Run every night at 02:00 and keep 8 weeks of reports
0 2 * * * /usr/bin/pgbadger -I -q -j 2 --retention 8 \
    -O /var/www/html/pgbadger /var/log/postgresql/postgresql-*.log
```

Step 8.4: View the reports

Browse to `http://your-server/pgbadger/` (after configuring Apache or Nginx to serve that folder and restricting access), or copy the directory to your workstation and open `index.html`.

Security note: Reports contain real query text, which can include sensitive data. Restrict web access with authentication or an internal network.

9. Read and interpret the report

Report section	What to look for
Overview	Total queries, peak queries per second, error counts, and the busiest time periods.
Top > Slowest individual queries	Single statements with the highest duration; first candidates for EXPLAIN ANALYZE.
Top > Time consuming queries	Queries with the highest total time; often the best tuning target.
Top > Most frequent queries	Hot queries; a small saving per call multiplies quickly.

Report section	What to look for
Queries by type / database / user / application	Which workloads (SELECT, INSERT, UPDATE) and clients drive the load.
Temporary files	Queries spilling to disk; consider raising work_mem or adding indexes.
Locks	Statements waiting on locks (needs log_lock_waits on).
Checkpoints	Frequency and write volume; frequent requested checkpoints suggest raising max_wal_size.
Autovacuum	Tables vacuumed and analysed most often and how long each takes.
Events	Errors, fatals and warnings grouped by message.
Connections / Sessions	Connection counts, session durations and connections per host.

Typical tuning workflow

- Open **Time consuming queries** and pick the top few.
- Copy the example query and run `EXPLAIN (ANALYZE, BUFFERS)` in psql.
- Add or adjust indexes, rewrite the query, or tune configuration.
- Generate a new report after the change and compare against the old one.

10. Troubleshooting

Problem	Likely cause and fix
Report is empty or shows 0 queries	log_min_duration_statement is -1 or too high, or log_line_prefix does not match. Check the log lines and pass -p.
"Unknown log format" or parse errors	Wrong format or prefix. Specify -f and -p explicitly.
Messages not recognised	lc_messages is not 'C'. Set it and reload.
Permission denied reading logs	Run as the postgres user, or add your user to the group that owns the log files.
Very slow processing	Use -j N and -J N, or limit the range with -b and -e.
Huge log files	Raise log_min_duration_statement, or enable logging only during analysis windows.
No lock, temp file or checkpoint sections	log_lock_waits, log_temp_files or log_checkpoints are not enabled.

11. Best practices and quick reference

- Do not leave `log_min_duration_statement = 0` on in production for long periods; the logging overhead and disk usage can be significant.
- Use log rotation (`log_rotation_age` / logrotate) and compress old logs; pgBadger reads .gz files directly.
- Keep the same `log_line_prefix` across servers so one command works everywhere.
- Use incremental mode with a retention policy for continuous monitoring.
- Protect generated reports, as they expose query text.

Minimum end-to-end recap

```
# 1. Install
sudo apt install pgbadger

# 2. Configure postgresql.conf (see section 4), then reload
sudo -u postgres psql -c "SELECT pg_reload_conf();"

# 3. Let the workload run, then generate the report
pgbadger -j 4 /var/log/postgresql/postgresql-*.log -o report.html

# 4. Open report.html in a browser
```

Official documentation and downloads: `https://github.com/darold/pgbadger`